

Flight Data Monitoring

IT4043E — Big Data Storage & Processing

Supervisor: Dr. Tran Viet Trung

Hanoi University of Science and Technology (SOICT)

January 2026

Authors:

Nguyen Duc Binh - 20225475

Tran An Khanh - 20225447

Pham Minh Hieu - 20220062

Truong Tuan Vinh - 20225464

Nguyen Nhu Giap - 20225441

Agenda

1. Motivation & problem statement
2. Data ecosystem
3. Architecture and component roles
4. Ingestion with NiFi
5. Streaming with Kafka
6. Processing with Spark
7. Dual storage: Cassandra (hot) & MinIO (cold)
8. Analytics layer: Trino + Superset
9. Deployment on Docker Compose and GKE
10. Results, challenges, and future work

Motivation & Problem

Why flight data needs a streaming + analytics pipeline

Aviation telemetry is high-frequency, global, and noisy.

We need low-latency processing for real-time monitoring
AND reliable storage for historical analytics and replay.

Goal: build a scalable reference architecture for real-time telemetry analytics using open-source components.

Big Data characteristics (5Vs)

- **Volume:** tens of thousands of concurrent flights → millions of events/day
- **Velocity:** updates every 1–10 seconds per aircraft
- **Variety:** semi-structured JSON with nested arrays (tracks)
- **Veracity:** missing coordinates, sensor noise, late arrival
- **Value:** density, trajectories, country-level statistics, weather correlation

Data Ecosystem

Primary telemetry + weather enrichment + static reference data

1) OpenSky Network

- Real-time states: /api/states/all (global snapshots)
- Historical flights: /api/flights/all (departure/arrival context)
- Tracks: /api/tracks/all (waypoint arrays for trajectories)

2) Aviation Weather Center (AWC) Data API

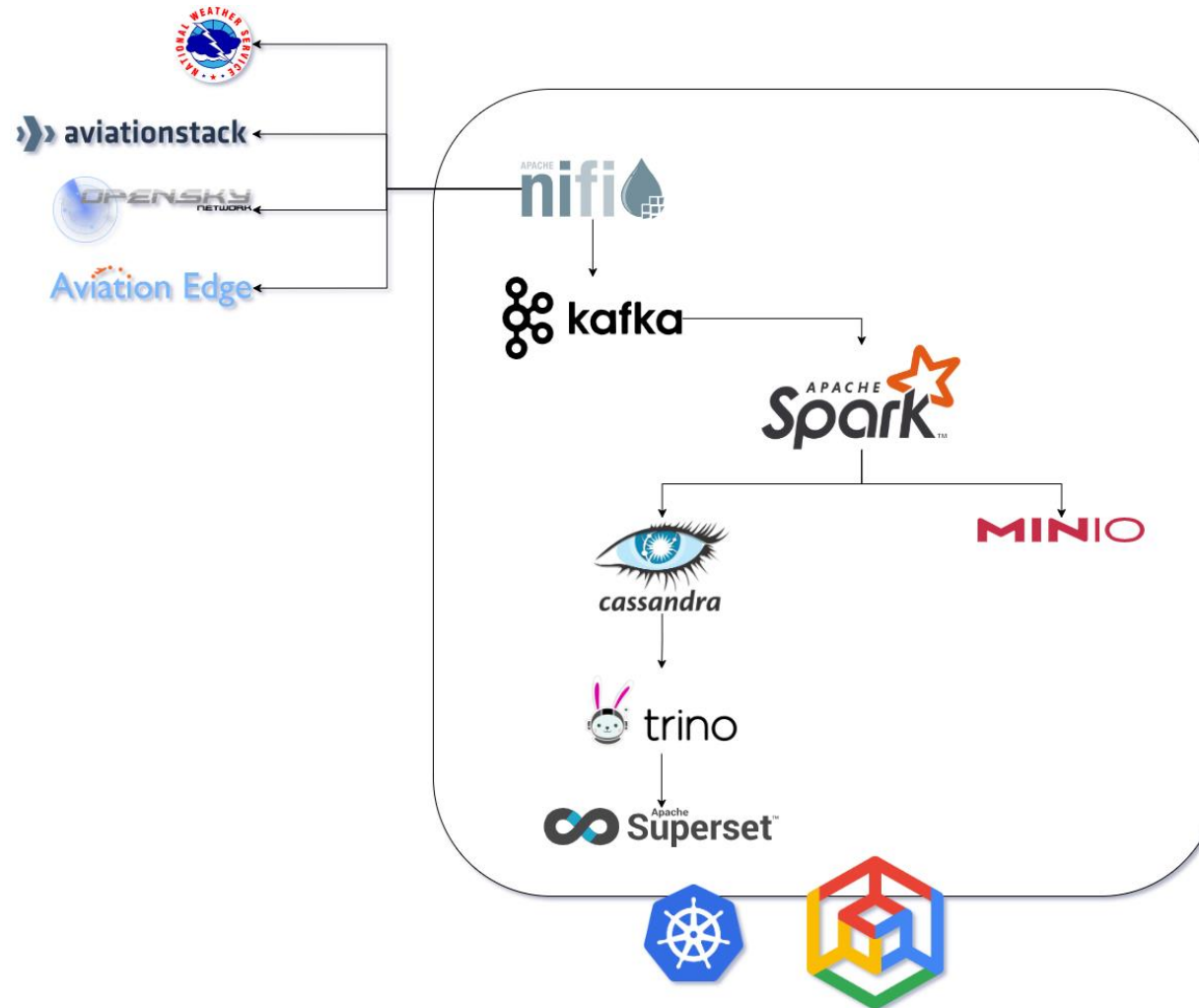
- METAR (observations) → current_airport_weather topic
- TAF (forecasts) → future_airport_weather topic
- SIGMET (hazards) → weather_warnings topic

3) Static reference data (Aviationstack)

- Airlines: IATA/ICAO codes, callsigns, country metadata
- Airports: coordinates, timezones, ISO codes
- Cities: city-level mapping for regional aggregations

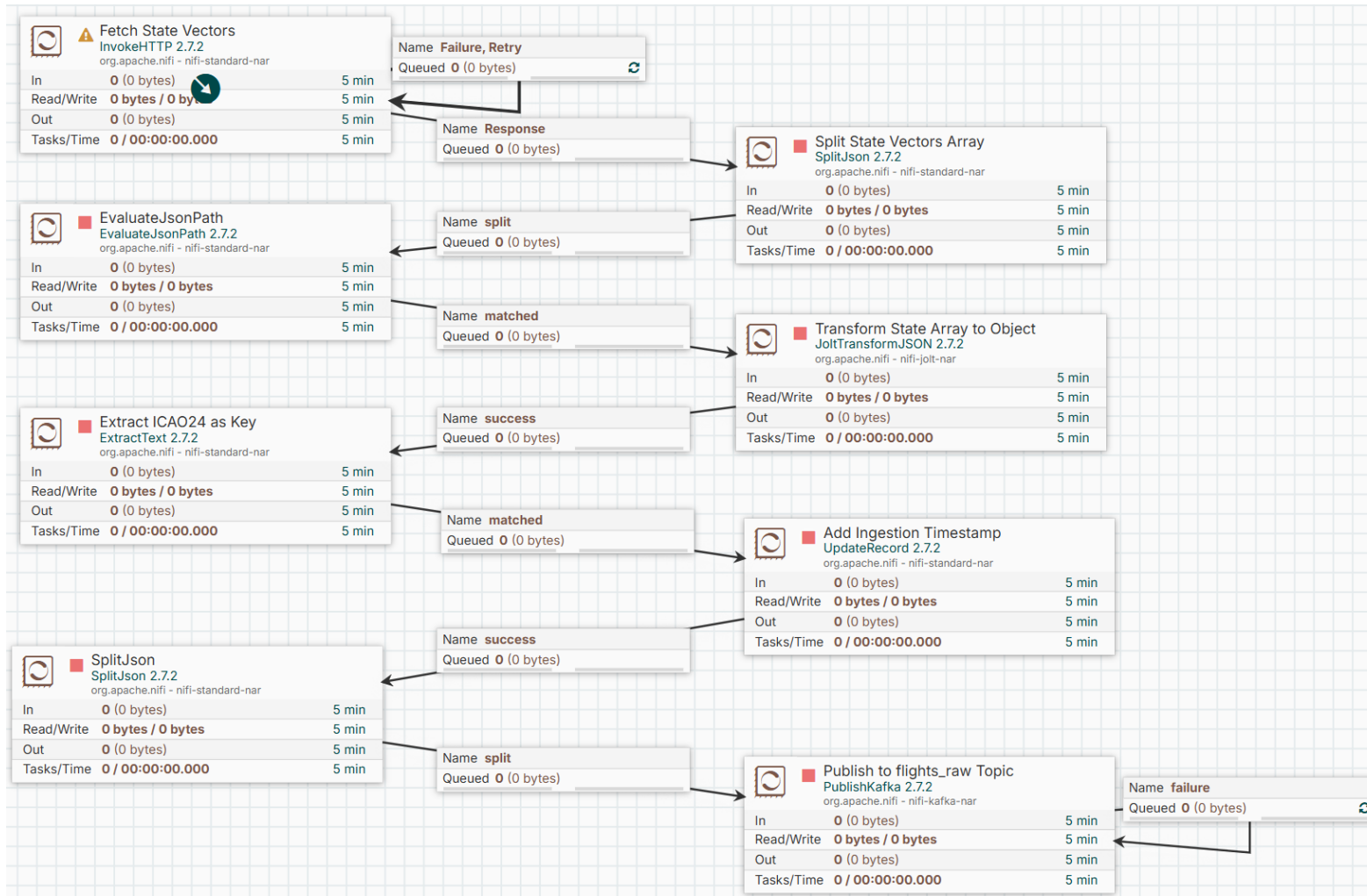
Overall Architecture

Kappa architecture: stream-first processing, dual storage, BI dashboards



Ingestion Layer: Apache NiFi (OpenSky)

Visual flow-based orchestration with backpressure and retries



Key flow mechanics

- OAuth2 token provider for higher API rate limits
- Polling intervals: states (20s), flights (15m), tracks (on-demand)
- SplitJson + JoltTransformJSON: array → structured JSON
- Kafka keys: icao24 (and icao24-startTime for tracks) to preserve ordering
- RouteOnAttribute: filter incomplete flight connections

Streaming Backbone: Apache Kafka

Durable, scalable transport between ingestion and processing

Kafka decouples producers (NiFi) and consumers (Spark), providing durability, replay, and horizontal scaling via partitions.

Topics (6) and semantics

- flights_raw (state vectors) • flight_data (arrival/departure info)
 - flight_track (trajectories)
- current_airport_weather • future_airport_weather • weather_warnings
- Retention: 7 days + Snappy compression (storage efficiency)

Reliability choices

- 3-broker cluster + ZooKeeper coordination
- Replication factor = 3 (topic & transaction state)
- Internal vs. Outside listeners for Docker/GKE connectivity

Processing: Spark Structured Streaming

Schema enforcement, enrichment, windowing, and multi-sink writes

Core responsibilities

- Consume Kafka streams with strict StructType schemas
- Clean & validate: nulls, NaNs/infs, whitespace trimming
- Geographical enrichment: country from airport or reverse geocode lat/lon
- Windowed aggregations + watermarking for late events
- Write concurrently to Cassandra (hot views) and MinIO (Parquet archive)

Windowing strategy

- 1-minute tumbling windows (live map) + 10s watermark
- 1-hour tumbling windows (statistics) + 10m watermark
- Append mode: only write final results after window closes

Example derived views (Cassandra)

- aircrafts_by_icao24 (latest state per aircraft)
- aircraftstates_by_icao24 (trajectory time-series)
- aircrafts_by_cell_minute (live density per geo-cell)
- activeaircraft_by_country_hour
- departures_by_country_hour

Performance notes

- Vectorized Pandas UDFs reduce Python ↔ JVM overhead
- approx_count_distinct (HyperLogLog) for scalable cardinality

Processing: Spark Structured Streaming

Spark Structured Streaming Dashboard for processing Network Flights

 3.4.0

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

Structured Streaming

StructuredNetworkFlight application UI

Spark Jobs (?)

User: spark
Total Uptime: 12 min
Scheduling Mode: FIFO
Active Jobs: 5
Completed Jobs: 4

▶ Event Timeline

▼ Active Jobs (5)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

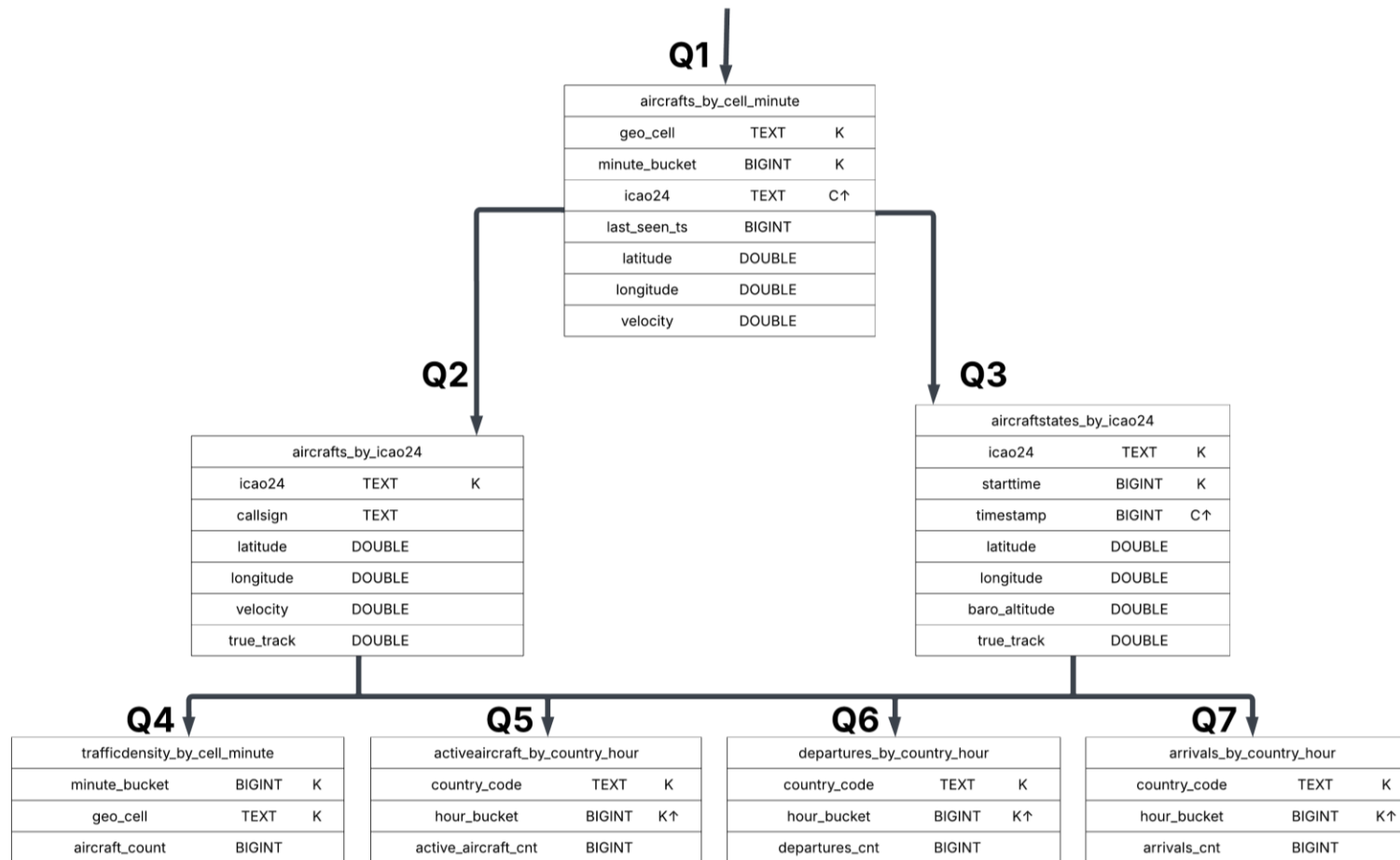
Job Id (Job Group) ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
8 (9fc53eff-9672-4746-bee3-90b32aa23697)	id = 29d6efa8-4532-4d3f-9010-4a433ff096d0 runId = 9fc53eff-9672-4746-bee3-90b32aa236... start at <unknown>:0 (kill)	2026/01/11 16:35:11	5.9 min	0/1	0/3
7 (55a4f8f2-0ff1-476d-9997-75e58e33cff5)	id = f02e7f91-6a47-4b6c-93fb-1cc843f0ec89 runId = 55a4f8f2-0ff1-476d-9997-75e58e33cff5 ... start at <unknown>:0 (kill)	2026/01/11 16:29:44	11 min	0/1	0/3
6 (f399bf05-06fc-49f3-a72d-af119ffcec3c)	id = 8bff96a9-bb99-4af7-bf95-3481180a8fa4 runId = f399bf05-06fc-49f3-a72d-af119ffcec3c ... start at <unknown>:0 (kill)	2026/01/11 16:29:44	11 min	0/1	0/3
5 (f6825e10-719b-438c-bc22-2851a1b5bce3)	id = d380ecaa-320c-42fa-a938-921d94291cc7 runId = f6825e10-719b-438c-bc22-2851a1b5b... start at <unknown>:0 (kill)	2026/01/11 16:29:40	11 min	0/2	0/203 (2 running)
4 (c70e365a-7e28-4ba5-9c14-fe3741e5a7e2)	id = eca54752-93de-40b5-a644-db9615ee8871 runId = c70e365a-7e28-4ba5-9c14-fe3741e5... start at <unknown>:0 (kill)	2026/01/11 16:29:40	11 min	1/2	3/203

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Completed Jobs (4)

Hot Storage: Apache Cassandra

Query-driven schema (Chebotko diagram) for low-latency dashboards



Design principles

- Query-first: denormalize into tables matching access patterns
- Avoid joins; use partition + clustering keys for fast reads
- Upserts give idempotency for reprocessing

Examples

- Latest snapshot: partition by icao24
- Trajectories: partition (icao24, startTime), cluster by timestamp
- Aggregations: (country_code, hour_bucket) to avoid hot partitions
- Consistency level ONE for low latency (AP preference)

Cold Storage: MinIO (S3-compatible)

Parquet archive for long-term analytics, replay, and checkpoints

MinIO acts as the data lake: raw + processed streams are written as Parquet with Snappy compression and time-based partitions (year/month/day).

Bucket organization

- flight-raw • flight-data • flight-tracks
- weather-metar • weather-taf • weather-isigmet
- checkpoints (Spark offsets + state for exactly-once recovery)

Write strategy

- High-velocity streams: 5-minute triggers (frequent archival)
- Lower-volume metadata: 10-minute triggers (larger files)
- Enables backfills: replay Kafka + rewrite Parquet partitions

Why keep a cold path?

- Cost-efficient long-term storage for analytics
- Auditable raw payloads (debug + veracity)
- Disaster recovery + reproducible processing
- Faster batch scans vs. Cassandra for history

Analytics Layer: Trino + Superset

SQL abstraction over Cassandra + interactive dashboards

Trino: distributed SQL engine

- Runs as Kubernetes-native coordinator + workers
- Cassandra connector with predicate pushdown (efficient filtering)
- Enables GROUP BY / aggregations beyond CQL limitations
- Exposed via NodePort for downstream tools

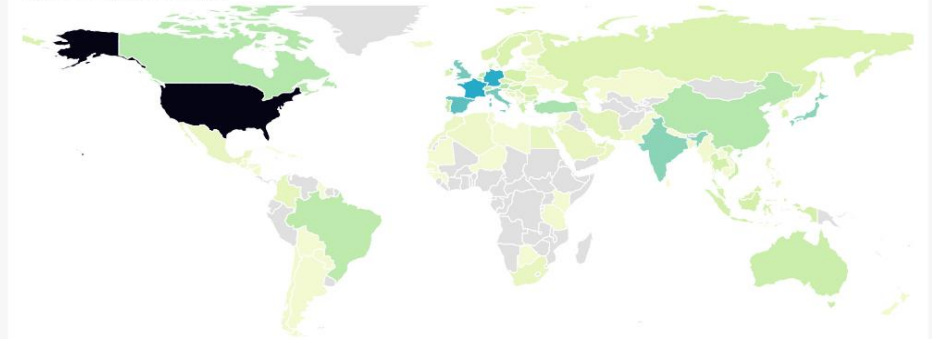
Superset: BI dashboards

- Trino-only connection via sqlalchemy-trino
- Redis caching for responsive UX (different TTLs)
- Semantic layer: datasets mapping Cassandra tables to metrics

Number of flights in 2026

15.1k
flights

Number of flights by country



Number of active aircrafts in 2026

976
aircrafts

Number of flights per hour



Analytics Layer: Trino + Superset

SQL abstraction over Cassandra + interactive dashboards

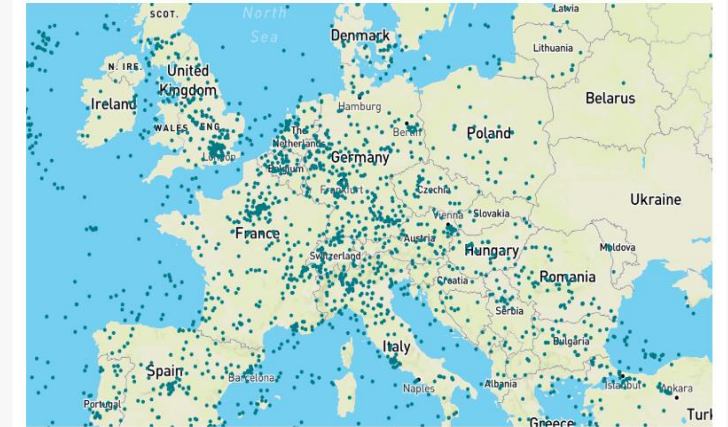
Superset: interactive exploration

- Interactive **maps** and **tables** for live and historical flight data
- Visual exploration of **routes**, **positions**, and **traffic density**
- Drill down from **overview** to **individual flights**
- Slice and filter by **time**, **location**, **altitude**, and **callsign**

Waypoints map



Live Flight Track



Waypoints table

Show entries

time_fromunix	callsign	lat	lon	on_ground	baro_altitude
2025-01-11 07:00:00	VN123	21.22	105.8	true	0
2025-01-11 08:10:00	VN123	15.5	107	false	35000
2025-01-11 10:00:00	VN123	1.36	103.99	true	0
2026-01-11 03:10:39	PNC0928	4.802999973297119	-74.03900146484375	false	2438
2026-01-11 03:10:50	PNC0928	4.7993998527526855	-74.03910064697266	false	2438
2026-01-11 03:10:57	PNC0928	4.7972002029418945	-74.03929901123047	false	2438
2026-01-11 03:11:15	PNC0928	4.791900157828467	-74.03980255126953	false	2438

Deployment & Operations

Local Docker Compose for fast iteration; GKE for scalable orchestration

You'll lose all free trial resources in 35 days. Upgrade to a paid account to keep your service running. You'll keep any remaining credit. Dismiss Upgrade

Google Cloud My First Project Search

Kubernetes Engine / Cluster: flight-data-gke / Nodes

All Fleets

No fleets in the current project

Resource Management

- Overview
- Clusters**
- Workloads
- AI/ML New
- Teams
- Applications
- Secrets & ConfigMaps
- Storage
- Object Browser
- Upgrades New
- Backup for GKE
- Marketplace
- Release Notes

Cluster details Edit Delete Deploy Connect Duplicate Equivalent code

flight-data-gke

Can't scale up a node pool because of a failing scheduling predicate Details

Overview Details **Nodes** Storage Observability Logs App Errors (38)

Node pools Create user-managed node pool

Filter Filter node pools

Name	Status	Version	Number of nodes	Machine type	Image type	Autoscaling	Default IPv4 Pod IP address
default-pool-v2	Ok	1.33.5-gke.2019000	2 (0 - 1 per zone)	e2-medium	Container-Optimized OS with containerd (cos_containerd)	0 - 1 nodes per zone	10.4.0.0/14
kafka-pool	Ok	1.33.5-gke.2019000	3 (1 per zone)	e2-standard-2	Container-Optimized OS with containerd (cos_containerd)	1 - 2 nodes per zone	10.4.0.0/14

Nodes

Local (Docker Compose)

- Fast feedback loop for development
- Shared Docker network + .env config
- Great for smoke tests, not for K8s primitives

GKE (production-like)

- Namespaces per component (Kafka, Spark, Trino, ...)
- StatefulSets + PVCs for Kafka/Cassandra
- Autoscaling node pools: stateful vs stateless
- Service exposure: LB (Superset), NodePort (NiFi/Trino), ClusterIP others

Deployment & Operations

Table 2: GKE Deployment Configuration per Component

Component	K8s Type	Replicas	CPU (req/lim)	Memory (req/lim)	Storage
Kafka	StatefulSet	3	500m / 1	1Gi / 2Gi	10Gi pd-ssd
ZooKeeper	StatefulSet	1	100m / 500m	256Mi / 512Mi	–
Cassandra	StatefulSet	1	300m / 1	1Gi / 2Gi	10Gi pd-ssd
Spark Streaming	Deployment	1	500m / 2	2Gi / 4Gi	–
NiFi	Deployment	1	500m / 2	2Gi / 3Gi	20Gi pd-balanced
MinIO	Deployment	1	100m / 500m	256Mi / 512Mi	20Gi standard-rwo
Trino Coordinator	Deployment	1	200m / 1	1Gi / 1.5Gi	–
Trino Workers	Deployment	3	300m / 1	1.5Gi / 2Gi	–
Superset	Deployment	1	200m / 500m	768Mi / 1.5Gi	–
Superset Postgres	StatefulSet	1	50m / 200m	128Mi / 256Mi	1Gi pd-balanced
Superset Redis	Deployment	1	–	–	–

Results & Takeaways

Delivered

- End-to-end pipeline from APIs → dashboards
- Near real-time monitoring with minute-level updates
- Historical trajectory + weather archiving in Parquet
- Exactly-once recovery using Spark checkpoints
- Deployable on local Docker & GKE

Challenges

- Python UDF overhead (CPU bottleneck)
- State management: checkpoints, TTL tuning
- Operational complexity across many components

Future work

- Predictive modeling (delays / congestion) with Spark MLlib
- Streaming anomaly detection for unusual trajectories
- Redis cache for reference data to reduce enrichment time
- HPA autoscaling based on consumer lag
- Security hardening: mTLS + secrets vault

Thank you!
Questions?